

OfferDesk

Две части одного репозитория.
Текст становится действием — и не врёт, когда не уверен.

Павел Кариков · 8 октября 2026

Сдаётся вариант 1 — персональный помощник «от текста до действия»

github.com/PavelKarikoff/OfferDesk

прод · 194.67.103.144:5001

ЧАСТЬ I · OFFERDESK

AI-квалификатор лидов ИЖС

Заявка или звонок → строгий JSON → сделка, грейд A/B/C, эскалация, черновик КП.

ЧАСТЬ II · PERSONAL ASSISTANT · ВАРИАНТ 1

От текста до действия

Входящая мысль → задача или заметка. Если модель не уверена — запись уходит на ручную проверку, без выдуманных полей.

Зачем две части

ЧАСТЬ I · ДЛЯ ОТДЕЛА ПРОДАЖ

30–40%

лидов «Дом-Мастер» остывают: норма реакции 15 минут, факт — часы. Одно КП вручную занимает 40–60 минут. Средний чек 8 млн ₽, маржа 15% — 1,2 млн ₽ с дома.

OfferDesk забирает сырой текст заявки, достаёт поля по схеме и отдаёт менеджеру сделку, которую можно проверить, а не переписывать с нуля.

ЧАСТЬ II · ДЛЯ ЧЕЛОВЕКА И МАЛЕНЬКОЙ КОМАНДЫ

текст → дело

Поручения живут в чатах и заметках: «позвонить», «не забыть», «идея». Часть так и не становится задачей.

Помощник кладёт текст в таблицу `items`, показывает его в панели и ставит `needs_review`, когда вход слишком общий или модель ответила плохо.

Общие слои: Flask, SQLite `deals.db`, OpenAI gpt-4o-mini, Pydantic, температура 0.2. Модель структурирует. Код решает, можно ли верить ответу.

Пять требований чеклиста

ТРЕБОВАНИЕ	ЧАСТЬ I · OFFERDESK	ЧАСТЬ II · ВАРИАНТ 1
1. Три точки API создать · витрина · ИИ	POST /deals/new — сделка в БД GET /deals — список POST /ingest — JSON DealExtraction	POST /capture — item в БД GET /tasks — список POST /tasks/extract — JSON без записи item
2. База и аудит каждый запуск	deals, users, action_log audit_log: вход, JSON, status, ошибка	items + audit_runs action, input, output, status, error, duration_ms, ts
3. Ручная проверка без выдумок	Эскалация: legal_risk, llm_failed, insufficient_data, validation_failed	needs_review и причина: LOW_CONFIDENCE, INVALID_JSON, SCHEMA_MISMATCH, AMBIGUOUS_INPUT
4. Три экрана	список /deals карточка сделки эскалация и журнал разбора	список /tasks карточка /tasks/<id> журнал /journal
5. Экономика на салфетке	1 операция: ~50 мин → ~6 мин. 100 операций: 41 500 ₽ → ~6 400 ₽. Ценность — скорость реакции, не только часы.	

Три точки квалификатора

01

СОЗДАТЬ

POST /deals/new

Менеджер вставляет транскрибацию или файл.
Пайплайн извлекает поля, считает эталон и пишет сделку в deals.

02

ВИТРИНА

GET /deals

Таблица рабочего места: клиент, бюджет, статус, можно ли собирать КП. Отсюда открывается карточка.

03

ИИ-ОБРАБОТКА

POST /ingest

Публичный JSON-вход. Возвращает DealExtraction, грейд и действие. Карточку сделки не создаёт — результат ложится в audit_log.

ЧТО ВОЗВРАЩАЕТ МОДЕЛЬ И ЧТО РЕШАЕТ КОД

Схема: клиент, объект, бюджет, финансирование, тон, уверенность, пустые поля. Дальше код, не модель: валидация, эскалация, грейд A/B/C, конверт action. Цена КП считается по прайсу компании.

Сделка пишется. Сомнение — тоже

ТАБЛИЦЫ

deals	Рабочая сущность: клиент, объект, бюджет, статус, эталон, источник разбора
audit_log	Каждый разбор: вход, JSON, source, status, ошибка, грейд, эскалация
action_log	Действия менеджера: создал, правил, собрал КП, отправил, сменил статус
users	Роли manager / admin

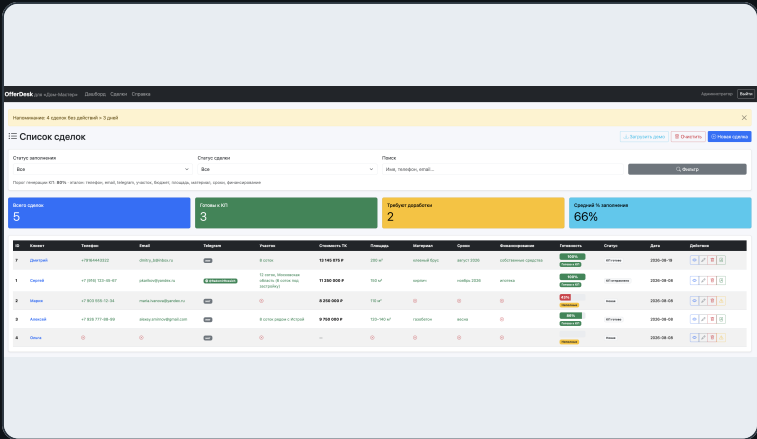
КОГДА ЧЕЛОВЕК ОБЯЗАТЕЛЕН

Эскалация не прячет слабый разбор. Сделка остаётся, но помечается причиной:

- legal_risk — суд, юрист, угроза в тексте
- llm_failed — модель недоступна, сработал regex
- insufficient_data — эталон ниже 30
- validation_failed — бюджет, площадь, телефон, почта

Безопасный контур: LLM упал → regex, процесс не останавливается, в журнал пишется ошибка. Выдуманная цена не появляется.

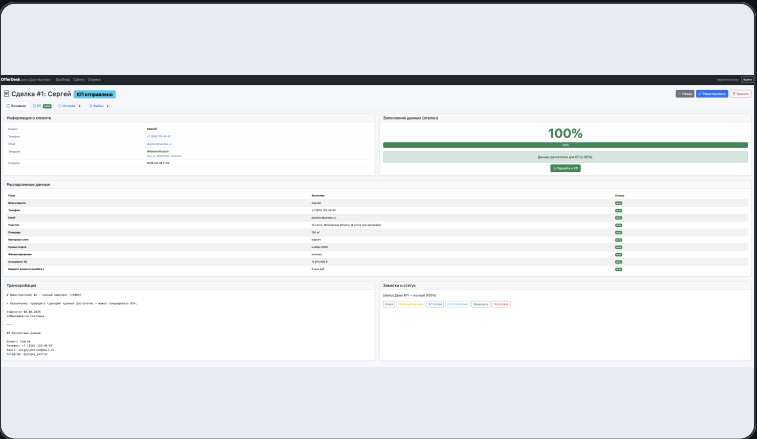
Три экрана менеджера



Витрина. /deals — очередь сделок, статусы, переход в карточку.

СПИСОК

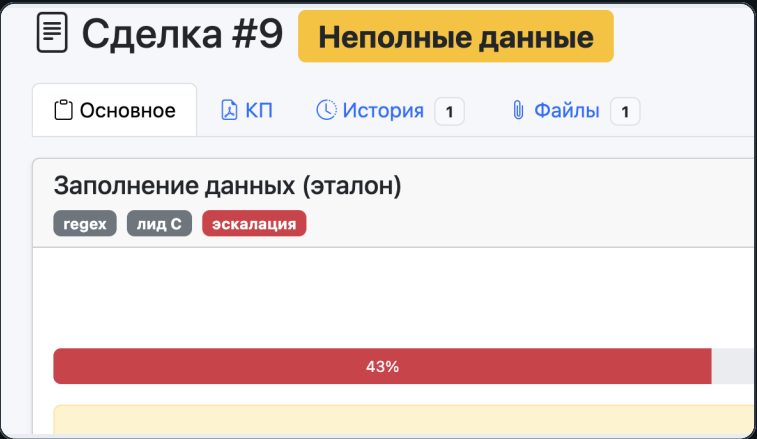
Очередь, а не чат. Видно, где данных хватает на КП, а где сделку надо добрать.



Карточка. Поля разбора, сырой протокол, эталон и действия: КП, статус, правка.

КАРТОЧКА

Поля разбора рядом с сырым протоколом. Менеджер сверяет, а не верит заголовку.



Проверка. Эскалация на карточке: лид С, эталон 43%. Причина пишется в audit_log.

ПРОВЕРКА

Слабый разбор не выглядит обычной сделкой: бейдж эскалации, низкий эталон, запись в журнале.

От текста до действия

01

СОЗДАТЬ

POST /capture

«Позвонить Сергею завтра в 10:00». Модель возвращает task, приоритет и срок. Строка появляется в items, запуск — в audit_runs.

СЦЕНАРИЙ

Вставил текст на /capture → увидел задачу в «Последних 5».

02

ВИТРИНА

GET /tasks

Фильтры «В работе», «Выполнено», «Требуется проверки», «Все». Кнопка «Выполнено» ставит статус done.

СЦЕНАРИЙ

Открыл /tasks, отфильтровал, закрыл выполненное.

03

ИЛИ БЕЗ ЗАПИСИ

POST /tasks/extract

Тот же разбор, но сущность не создаётся. Ответ — строгий JSON. В аудит пишется запуск с пустым item_id.

СЦЕНАРИЙ

Красный бейдж → карточка → правка заголовка → снятие с проверки.

Система не делает вид, что уверена

ITEMEXTRACTION · TEMPERATURE 0.2

```
{
  "item_type": "task | note",
  "title": "≤ 80 символов",
  "priority": "low | medium | high",
  "due_date": "YYYY-MM-DD | null",
  "tags": ["..."],
  "confidence": 0.0,
  "needs_review": false,
  "review_reason": null
}
```

Пустой текст → HTTP 400, модель не вызывается. Дата «завтра» считается от строки «Сегодня: YYYY-MM-DD» в промпте.

LOW_CONFIDENCE

confidence < 0.5

INVALID_JSON

Ответ не JSON или модель упала

SCHEMA_MISMATCH

JSON не прошёл Pydantic

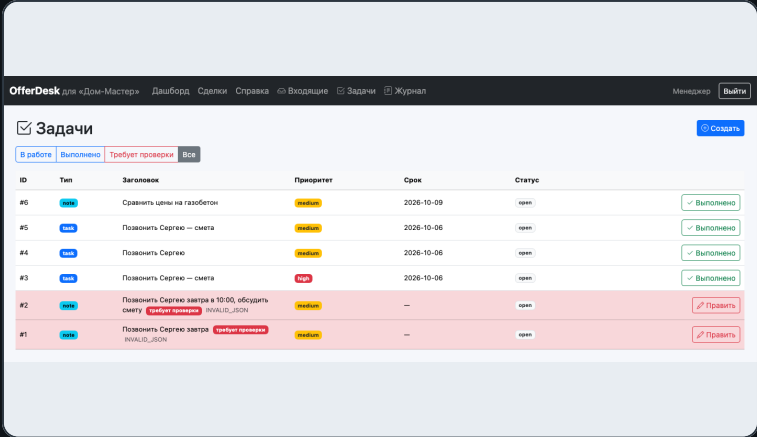
AMBIGUOUS_INPUT

«сделай важное», слишком общий вход

БЕЗОПАСНЫЙ ОТВЕТ · ПРОГОН BAD_01

Вход «сделай важное» → заголовок остаётся «сделай важное», confidence 0.00, источник heuristic, причина AMBIGUOUS_INPUT. Срок и теги не выдуманы.

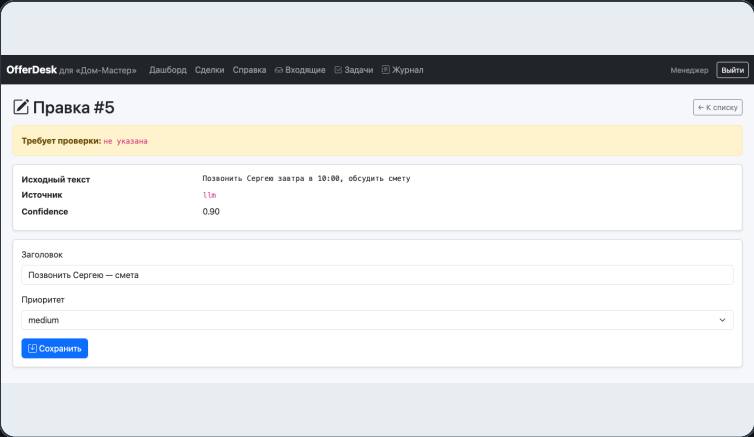
Список, карточка, журнал



Витрина /tasks. Красные строки – «требуется проверки», фильтр отдельный.

ВИТРИНА

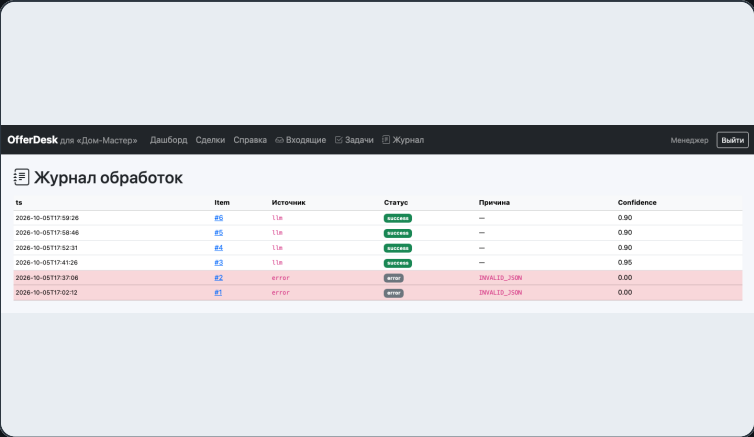
Фильтр «Требуется проверки» – отдельный список. Красная строка не смешивается с обычными задачами.



Карточка /tasks/<id>. Сырой текст, источник, confidence, правка заголовка.

КАРТОЧКА

Сырой текст, источник и confidence. Правка заголовка снимает запись с проверки.



Аудит /journal. Время, статус, причина, confidence каждого запуска.

ЖУРНАЛ

Каждый запуск: success или error, причина, confidence. Сбой виден, а не затирается.

100 операций: 50 минут против 6

ДО · 1 ОПЕРАЦИЯ ~50 мин 15–20 мин разобрать заявку и 40–60 мин собрать КП.	ПОСЛЕ · 1 ОПЕРАЦИЯ ~6 мин Вставить текст и проверить то, что система уже разложила.	100 ОПЕРАЦИЙ · ДО 41 500 ₽ ~83 часа × 500 ₽/час.	100 ОПЕРАЦИЙ · ПОСЛЕ ~6 400 ₽ 5 000 ₽ проверка + ~700 ₽ OpenAI + ~700 ₽ VPS.
ЭКОНОМИЯ ВРЕМЕНИ ~35 000 ₽ на каждые 100 операций, если считать только часы людей.	БАЗОВЫЙ СЦЕНАРИЙ ~1 неделя Высвобождение часов плюс одна лишняя сделка: 1,2 млн ₽ маржи.	МАСШТАБ API 0,25% Бюджет ~100 разборов — четверть процента маржи одного дома.	
Ценность не в сэкономленных минутах. Заявка не остывает: ответ за минуты вместо часов даёт +10–15% к замеру. Помощник при этом не штампует уверенные задачи из мусорного текста.			

Где модель сильнее правил — и где человек остаётся

GOLDEN SET · 15 ПРОТОКОЛОВ · 30.09.2026

ПОЛЕ	REGEX	LLM V1 В ПРОДЕ
Exact эталон	8/15 · 53%	14/15 · 93%
budget_rub	0.00	1.00
start_date	0.30	1.00
financing	0.50	1.00
area_m2	0.54	0.96

Промпт v2 (few-shot, снимок 29.09) — 15/15. В проде сейчас v1. Пороги: эталон ≥ 80% → КП, 50–79% → дозаполнить, < 50% → перезвонить.

10 ВХОДОВ ПОМОЩНИКА · REPORTS/ITEM_TESTS.MD

9 llm 2

из 10 разобраны моделью ушли на проверку

Оба спорных входа — AMBIGUOUS_INPUT: шум и «сделай важное». Четыре задачи получили срок и приоритет. Четыре заметки не превратились в фальшивые дедлайны.

- Температура извлечения 0.2 — схема, не сочинение.
- Цену КП считает прайс, не модель.
- Нет OpenAI — regex или эвристика, флаг проверки, запись ошибки. Процесс не останавливается.

Что можно показать за минуту

ПРОБЛЕМА

Отдел теряет лиды, пока менеджер вручную разбирает звонок и собирает КП. Рядом теряется поток коротких поручений — они не доходят до списка дел.

РЕШЕНИЕ

Один сервис, две схемы JSON. Квалификатор ведёт сделку. Помощник ведёт задачу. Оба пишут аудит и отдают сомнительное человеку, а не «додумывают».

РЕЗУЛЬТАТ

Три точки на каждую часть. Шесть таблиц. Три экрана панели у каждой части. Около 35 000 ₽ на 100 операций и неделя до окупаемости, если приходит одна лишняя сделка.

РЕПОЗИТОРИЙ

github.com/PavelKarikoff/OfferDesk

релиз v1.2.2-final-checklist

ПРОД

194.67.103.144:5001

/deals · /capture · /tasks · /journal

ПАВЕЛ КАРИКОВ

Спасибо. Вопросы?